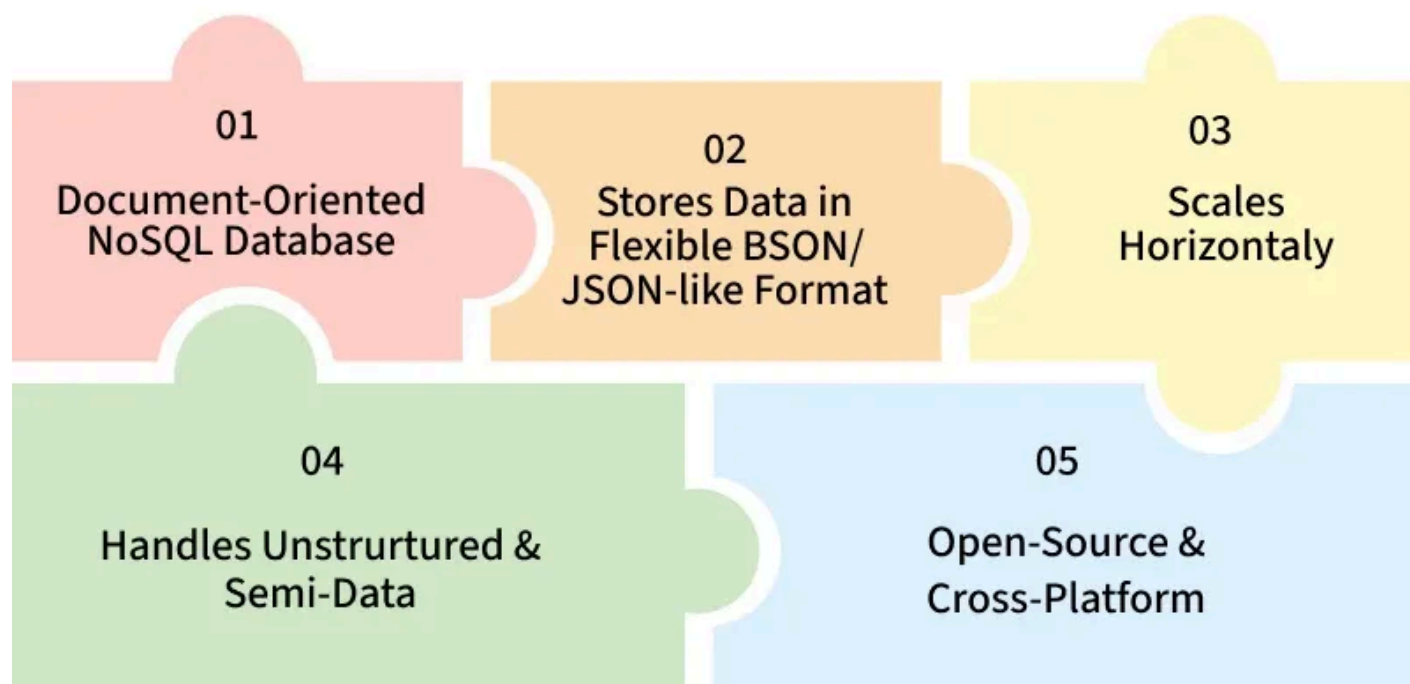


Chapter 3 Popular NoSQL Databases

1. Mongo DB

MongoDB is a popular document-oriented NoSQL database that stores data in a flexible BSON format, enabling fast and efficient storage and retrieval. It is available under the SSPL, which is not recognized as an open-source license by the Open Source Initiative due to commercial use restrictions.



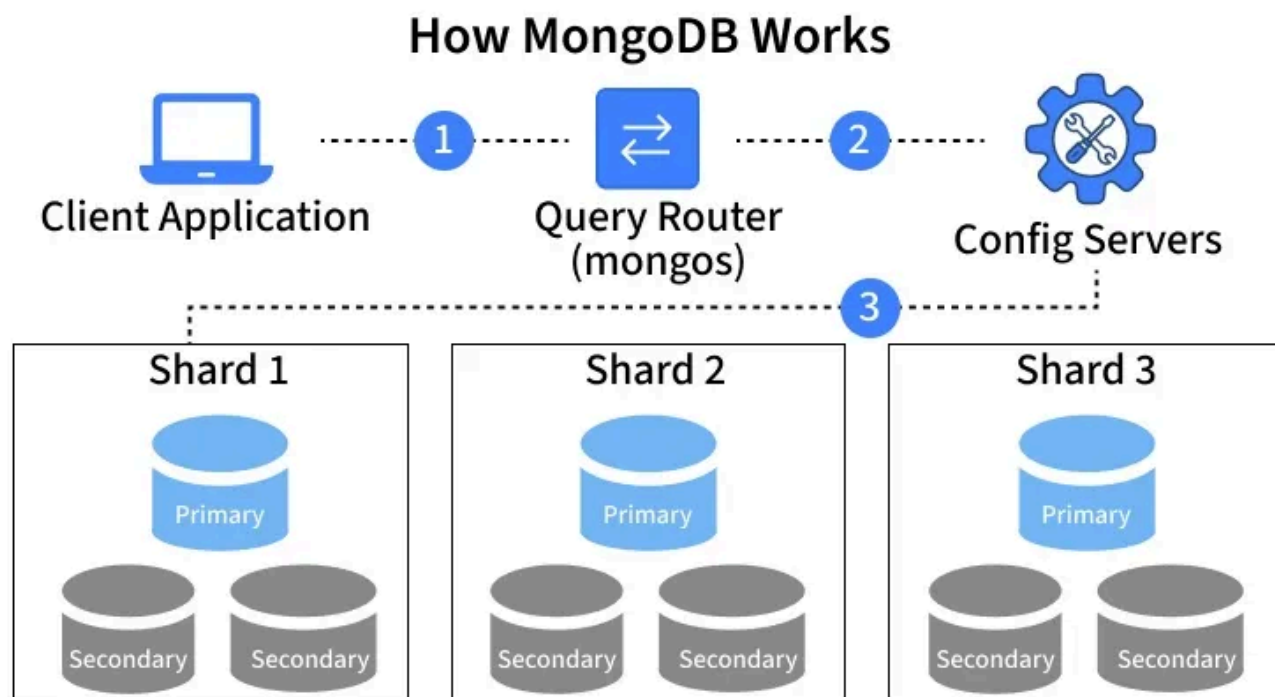
A simple MongoDB document Structure:

```
{
  "_id": 1,
  "title": "Geeksforgeeks",
  "author": "John Smith",
  "url": "https://www.geeksforgeeks.org/",
  "tags": ["NoSQL", "Database", "Tutorial"],
  "published": true,
  "views": 1500
}
```

- **_id:** Unique identifier for the document.
- **title, author, url:** Basic fields storing strings.
- **tags:** An array of strings.
- **published:** A Boolean value.
- **views:** A number representing views or count.

1.1 Working of MongoDB

In a MongoDB sharded cluster, the components work together as follows:



- **Client Application:** The application sends requests to MongoDB to read or write data.
- **Query Router (mongos):** The query router receives the client request and determines which shard should handle it.
- **Config Servers:** Store metadata about the cluster, including which data resides on which shard.
- **Shards:** Data is horizontally distributed across replica sets, with the primary handling writes and secondaries providing replication and reads.
- **Primary Node:** Handles all write operations.
- **Secondary Nodes:** Replicate data from the primary for high availability and read operations.

1.2 CRUD Operations

MongoDB provides several basic operations to manage and manipulate data efficiently:

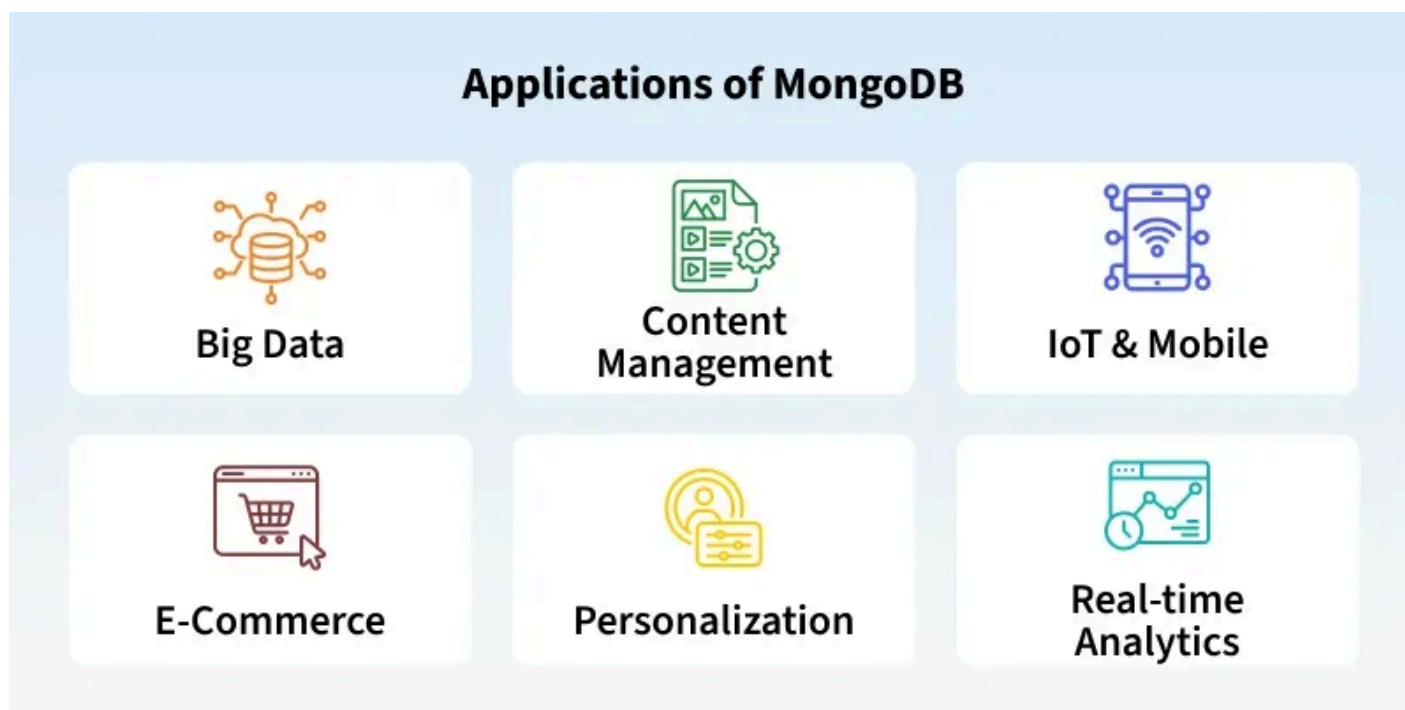
- **Create (Insert):** Add new documents to a collection.
- **Read (Find):** Retrieve documents from a collection based on query criteria.
- **Update:** Modify existing documents in a collection.
- **Delete:** Remove documents from a collection.

1.3 Features of the MongoDB database

MongoDB offers several key features that make it efficient and scalable:

- **Document Oriented:** Stores all related data together in a single, flexible document rather than multiple tables.
- **Indexing:** Enables fast queries by avoiding full collection scans, crucial for handling large data efficiently.
- **Scalability:** Uses sharding to distribute data across multiple servers; easily add new machines to expand.
- **Replication and High Availability:** Stores multiple data copies on different servers for redundancy and fault tolerance.
- **Aggregation:** Processes and summarizes data (like SQL GROUP BY) using operations such as sum, avg, min, and max.

1.4 Real-World Applications of MongoDB



MongoDB powers a wide range of modern applications:

- **Websites & Apps:** Platforms like e-commerce sites use MongoDB to store product catalogs, user profiles, and order histories.
- **Real-Time Analytics:** Companies analyze user activity or sales trends on the fly, such as tracking popular products during a sale.
- **Content Management & Personalization:** Apps manage dynamic content and personalize recommendations, like suggesting movies based on viewing history.
- **Big Data & IoT:** MongoDB handles large volumes of unstructured data from sensors, logs, or social media feeds for processing and insights.

1.5 Language Support by MongoDB

MongoDB offers official drivers for many popular programming languages, enabling easy integration and database operations.

- C
- C++
- Rust
- C#
- Java
- Node.js
- Perl
- PHP
- Python
- Ruby
- Scala
- Go
- Erlang

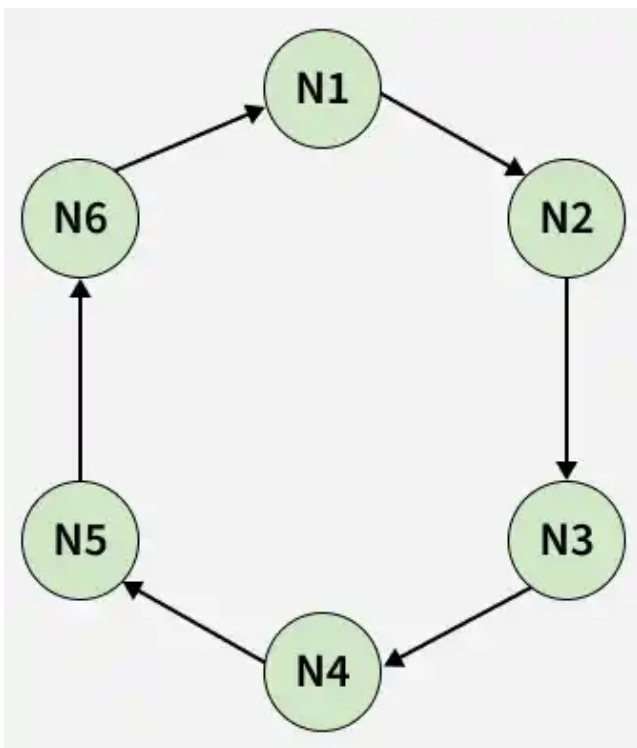
2. Apache Cassandra (NoSQL Database)

Apache Cassandra is an open-source, distributed NoSQL database designed to handle large volumes of data across multiple servers. Apache Cassandra has the capability to handle structured, semi-structured, and unstructured data.

- Cassandra supports **ACID** properties at the single-partition (single-row) level.
- It is scalable, highly available and also **fault-tolerant**.

- It does not support full multi-row or multi-table ACID transactions.

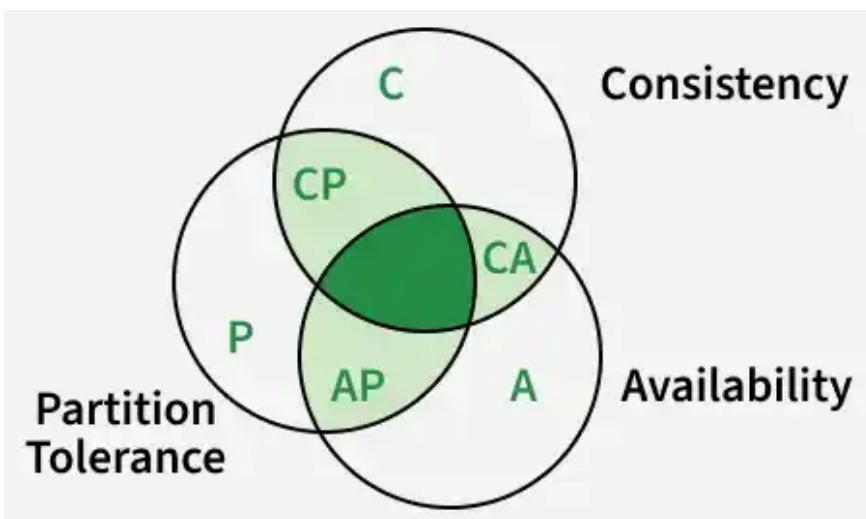
Masterless Ring Architecture of Cassandra



Masterless Ring Architecture of Cassandra

Apache Cassandra uses a masterless peer-to-peer architecture, where all nodes are equal and there is no single point of failure. In Apache Cassandra, multiple replicas of data can be created by configuring the replication strategy and Replication Factor (RF) when creating a keyspace.

CAP Theorem



CAP Theorem

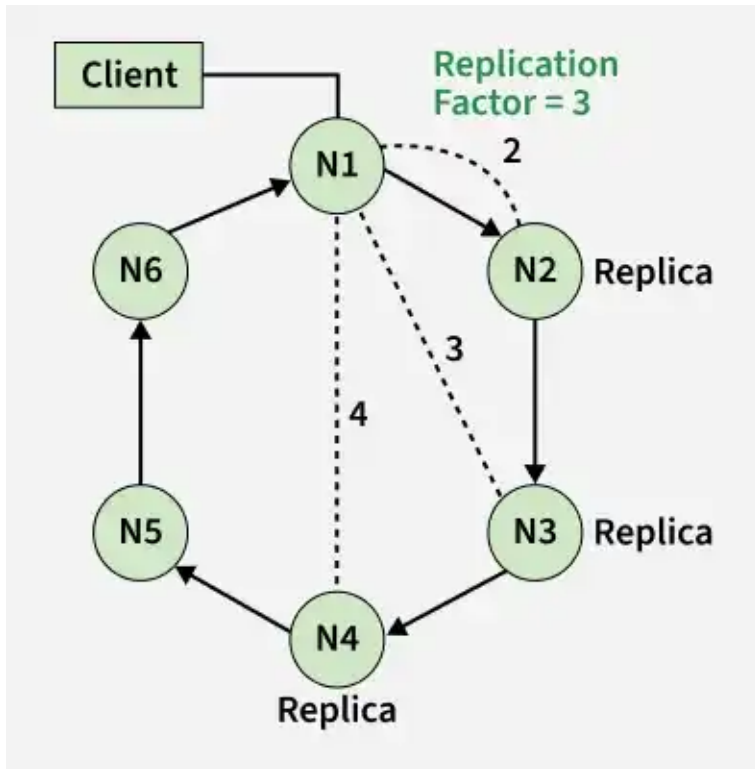
It provides high availability and partition tolerance while offering tunable consistency, allowing applications to balance consistency and availability based on their requirements.

Example:

```

CREATE KEYSPACE Example
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 3
};
  
```

In this example, Replication Factor (RF) = 3 means Cassandra stores three replicas of each partition on different nodes according to the configured replication strategy.



RF = 3

cqlsh: cqlsh is the command-line shell used to interact with Apache Cassandra using Cassandra Query Language (CQL)

CQL query for Basic Operation:

Step 1: To create keyspace use the following CQL query.

```
CREATE KEYSPACE Emp
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
};
```

Step 2: Use the following CQL command to select the keyspace.

Syntax:
USE keyspace-name
USE Emp;

Step 3: To create a table use the following CQL query.

Example:
CREATE TABLE Emp_table (
 name **text** **PRIMARY KEY**,
 Emp_id **int**,
 Emp_city **text**,
 Emp_email **text**
);

Step 4: To insert into Emp_table use the following CQL query.

```
Insert into Emp_table(name, Emp_id, Emp_city, Emp_email)
VALUES ('Ashish', 1001, 'Delhi', 'ashish05.rana05@gmail.com');

Insert into Emp_table(name, Emp_id, Emp_city, Emp_email)
VALUES ('Ashish Gupta', 1001, 'Bangalore', 'ashish@gmail.com');

Insert into Emp_table(name, Emp_id, Emp_city, Emp_email)
VALUES ('Amit', 1002, 'Noida', 'abc@gmail.com');

Insert into Emp_table(name, Emp_id, Emp_city, Emp_email)
VALUES ('Dhruv', 1003, 'Pune', 'xyz@gmail.com');
```

```

Insert into Emp_table(name, Emp_id, Emp_city, Emp_email)
VALUES ('Shivang', 1004, 'Mumbai', 'test@gmail.com');

Insert into Emp_table(name, Emp_id, Emp_city, Emp_email)
VALUES ('Aayush', 1005, 'Gurugram', 'cass_write@gmail.com');

Insert into Emp_table(name, Emp_id, Emp_city, Emp_email)
VALUES ('Bhagyesh', 1006, 'Chandigarh', 'welcome@gmail.com');

```

Step 5: To read data, use the following CQL query.

```

SELECT * FROM Emp_table;

```

3. Redis

Redis (Remote Dictionary Server) is an in-memory database that stores data in RAM for very fast access. It is mainly used for caching and reducing the load on the main database.

- Stores frequently accessed data for faster retrieval and better application performance.
- Used for sessions, queues, leaderboards, and real-time analytics.

Example: An e-commerce website can use Redis to cache product details so users can load product pages much faster.

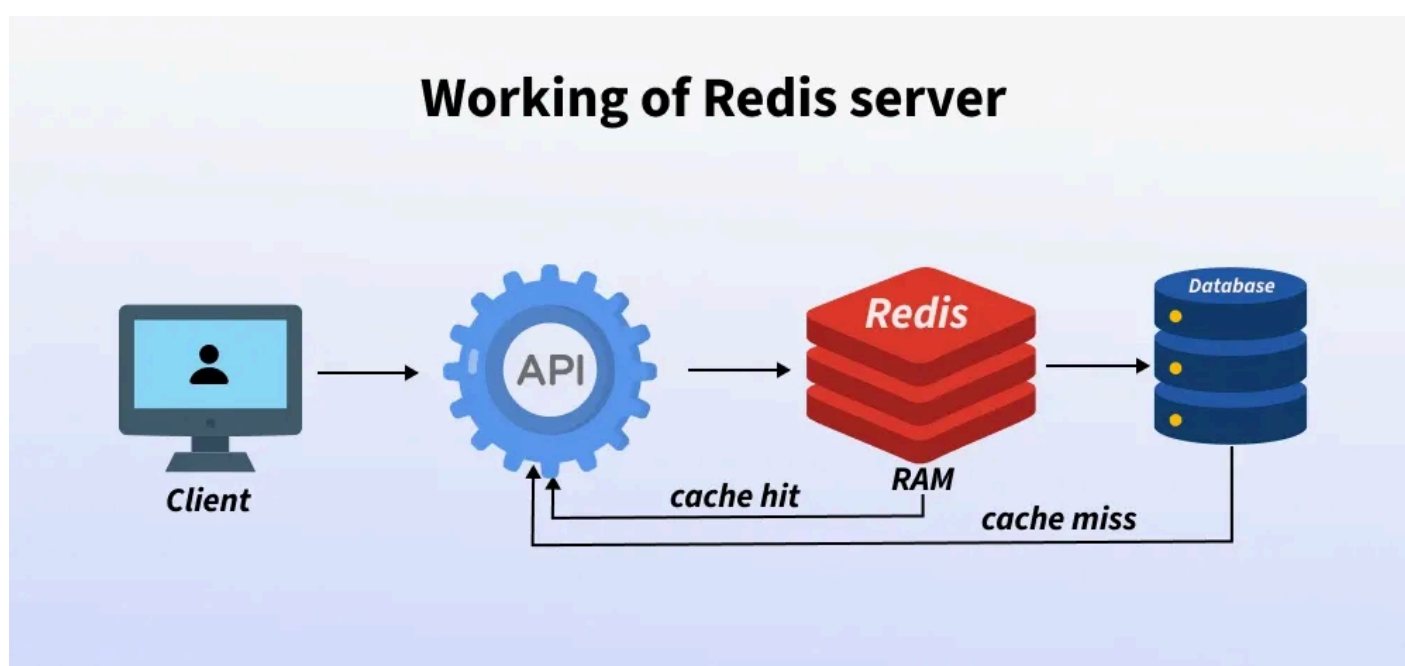
3.1 Real-World Applications

Redis is widely used by large-scale applications to handle high-speed data access, real-time processing, and efficient caching.

- **Amazon & Flipkart:** Use Redis to cache product details, prices, and user sessions, ensuring fast page loads and smooth checkout during high traffic sales.
- **Netflix:** Uses Redis for caching frequently accessed content data and managing real-time user sessions to deliver a seamless streaming experience.
- **Facebook & Instagram:** Use Redis to handle real-time notifications, feeds, and user activity for fast and responsive interactions.
- **Uber:** Uses Redis for real-time location tracking, ride matching, and surge pricing calculations.

3.2 Working

Redis acts as a caching layer between the database and the client to speed up data access and reduce the load on the main database. When a client asks for data, the API Gateway forwards the request to Redis.



- **Request Handling:** The client request is routed through the API Gateway, which first checks Redis for the required data.
- **Cache Hit:** If the data exists in Redis, it is returned immediately without accessing the database.
- **Cache Miss:** If the data is not found in Redis, the request is forwarded to the main database.
- **Cache Update:** The data retrieved from the database is stored in Redis for future requests.
- **Response to Client:** The API Gateway sends the final response to the client using data from Redis or the database.

Example: When a user visits a popular product page, Redis serves the cached product information instantly instead of querying the database every time.

```
import redis
r = redis.Redis(host='localhost', port=6379, db=0)

r.set('name', 'Alia')
print(r.get('name').decode('utf-8'))

r.set('name', 'Riya')
print(r.get('name').decode('utf-8'))

r.delete('name')
print(r.get('name'))
```

Output:-

```
None
(.venv) gfg0622@Anshu-ka-lappi:~/Downloads$ ./
Alia
Riya
None
(.venv) gfg0622@Anshu-ka-lappi:~/Downloads$
```

```
import redis
```

This line imports the `redis` Python library, which allows you to talk to a Redis server from your Python code.

```
r = redis.Redis(host='localhost', port=6379, db=0)
```

This line creates a connection to the Redis server.

- **host='localhost':** Connects to the Redis server running on your local machine.
- **port=6379:** Uses the default port on which Redis listens for connections.
- **db=0:** Connects to Redis logical database 0. Redis supports multiple logical databases numbered from 0 onward.

The variable `r` acts as a Redis client object that can be used to perform various Redis operations.

```
r.set('name', 'Alia')
```

This statement stores a key-value pair in Redis.

- **Key:** name
- **Value:** Alia

After execution, Redis stores the data as:

```
name -> Alia
```

This line stores the key `name` with the value `Alia` in Redis.

```
print(r.get('name').decode('utf-8'))
```

This statement retrieves and displays the value associated with the key name.

- `r.get('name')` fetches the value from Redis.
- Redis returns data in bytes format, such as `b'Alia'`.
- `.decode('utf-8')` converts the bytes into a readable string.
- The output displayed on the console is:

Note: In this example, Redis is integrated with Python but redis can also be implemented in other programming languages like Java, JavaScript (Node.js), Go, Ruby, C# (.NET), PHP, and many more, using their respective Redis client libraries.

3.3 Use Cases of Redis

Redis is used when applications need very fast access to temporary or frequently used data, reducing repeated queries to the main database.

- **Caching:** Stores frequently accessed data for faster retrieval and improved performance.
- **Session Management:** Stores user login and session information for quick access.
- **Real-time Applications:** Supports leaderboards, queues, notifications, and chat systems with fast updates.

Example: In a messaging application, Redis can store the last few messages of a conversation using its list data structure so that users can quickly see recent messages without repeatedly querying the main database.

3.4 Factors That Make Redis Fast

Redis is fast because it stores data in RAM and uses an efficient architecture for quick data processing.

- **In-Memory Storage:** Stores data in RAM for faster read and write operations.
- **Single-Threaded Event Loop:** Avoids thread management and context-switching overhead.
- **Efficient Data Structures:** Uses optimized lists, sets, hashes, and sorted sets.
- **Lightweight Protocol:** Uses RESP for fast client-server communication.

Example: When an application requests cached product data from Redis, the response can be returned in microseconds, whereas querying a disk-based database may take milliseconds.

4. CouchDB

CouchDB is an open-source, document-oriented NoSQL database developed by the Apache Software Foundation. CouchDB stores data as JSON documents containing key-value pairs, lists, and maps.

- **Flexible Document Model:** Stores data in schema-less JSON documents.
- **Powerful Querying:** Supports data filtering, mapping, and querying.
- **RESTful API:** Uses an HTTP-based REST API for easy database communication.
- **Simple HTTP Methods:** Supports standard methods like GET, PUT, and DELETE for CRUD operations.

4.1 CouchDB Data Model

- In CouchDB each database is a collection of independent documents which makes the outermost data structure/container of it.
- Each document maintains its own data and self-contained schema.

- To merge the differences occurred while the databases were disconnected, Document metadata contains revision information.
- To avoid the need to lock the database field during writes, it implements multi version concurrency control.

Simple CouchDB Document Structure

```
{
  "_id": "101",
  "name": "John Doe",
  "age": 25,
  "email": "john@example.com",
  "city": "Mumbai"
}
```

- **_id:** Unique identifier for the document.
- **Fields:** Store data as key-value pairs (e.g., name, age, email, city).
- **JSON Format:** Documents are stored in JSON format.
- **Flexible Schema:** Different documents can have different fields.

4.2 Advantages

- **Schema-less Database:** Stores JSON documents without requiring a fixed schema.
- **Easy Replication:** Supports master-master replication and data synchronization across multiple servers.
- **RESTful API:** Uses HTTP/HTTPS, making it easy to integrate with web applications.
- **Offline-First Support:** Allows applications to work offline and synchronize data when connectivity is restored.

4.3 Limitations

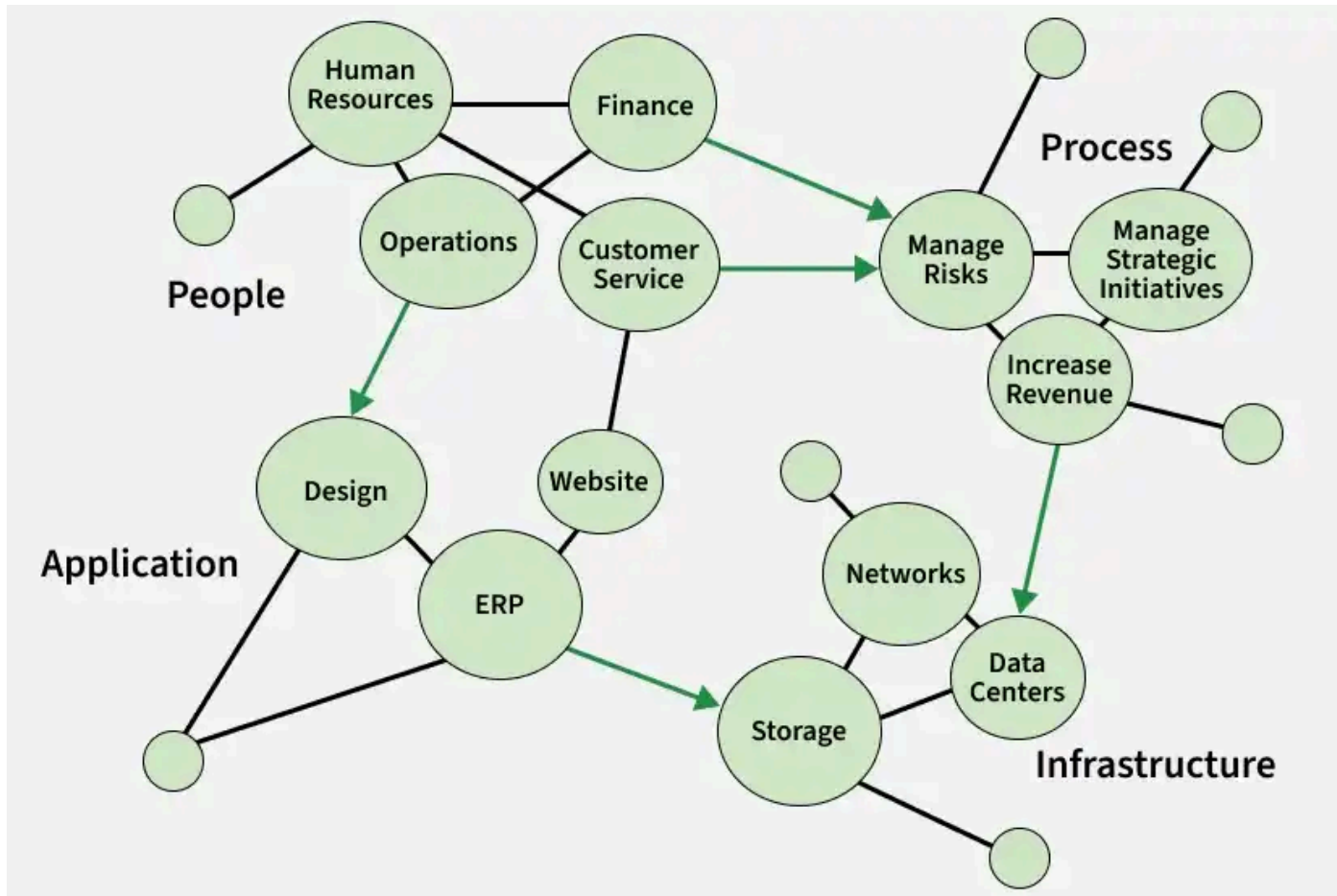
- **No Native Joins:** Does not support SQL-style joins between documents.
- **Limited Querying:** Complex queries are harder compared to relational databases.
- **Eventual Consistency:** Replicated data may not be immediately consistent across all nodes.
- **Not Suitable for Highly Relational Data:** Performs poorly for applications with many interconnected relationships

5. Neo4j : The Graph Database

Neo4j is a leading open-source graph database management system and a NoSQL database designed to store, manage and query highly connected data efficiently. Unlike traditional relational databases such as MySQL, which organize data in tables. This graph-based approach enables fast traversal of complex relationships.

- Nodes represent entities such as people, products, locations or any other data objects.
- Relationships connect nodes and define how the entities are related to each other.
- Properties are key-value pairs that store additional information about nodes and relationships.

Graph Database



Graph Database

The diagram shows that People use Applications and IT Infrastructure to execute Business Processes, helping the organization achieve goals like managing risks and increasing revenue.

- **People:** Includes departments like Human Resources, Finance, Operations and Customer Service. These teams perform business activities.
- **Process:** Business processes such as Manage Risks, Manage Strategic Initiatives and Increase Revenue help the organization achieve its objectives.
- **Application:** Software systems like ERP, Website and Design applications support employees and automate business operations.
- **Infrastructure:** IT resources such as Networks, Storage and Data Centers provide the technical foundation for applications to run.

5.1 Neo4j Architecture

Neo4j consists of the following main components:

- **Storage Engine:** Stores graph data efficiently.
- **Query Engine:** Executes and optimizes Cypher queries.
- **Transaction Manager:** Ensures ACID properties and data consistency.
- **Index Manager:** Speeds up node searches using indexes.
- **Cluster Manager:** Provides replication, high availability and fault tolerance.

5.2 Features of Neo4j

- **High Performance and Scalability:** Neo4j is designed to handle massive amounts of data and complex queries quickly and efficiently. Its native graph storage and processing engine ensure high performance and scalability, even with billions of nodes and relationships.
- **Cypher Query Language:** Neo4j uses Cypher, a powerful and expressive query language tailored for graph databases. Cypher makes it easy to create, read, update and delete data, allowing users to perform complex queries with concise and readable syntax.
- **ACID Compliance:** Neo4j ensures data integrity and reliability through ACID (Atomicity, Consistency, Isolation, Durability) compliance. This guarantees that all database transactions are processed reliably and ensures the consistency of the database even in the event of failures.

- **Flexible Schema:** Unlike traditional databases, Neo4j offers a flexible schema, allowing users to add or modify data models without downtime. This adaptability makes it ideal for evolving data structures and rapidly changing business requirements.

5.3 Core Concepts of Neo4j

Neo4j stores data using three main components:

1. Nodes: Nodes represent entities or objects such as Person, Customer, Product, City or Movie.

```
Example: Person {name: "Alice", age: 28}
```

2. Relationships: Relationships connect two nodes. They have a direction, a type and can store properties.

```
Example: (Alice)-[:FRIEND_OF]->(Bob)
```

3. Properties: Properties are key-value pairs that store information about nodes or relationships.

Examples:

- Node: Book {title: "Database Systems", price: 850}
- Relationship: (Alice)-[:PURCHASED {date: "2026-06-01"}]->(Laptop)

5.4 Cypher Query Language

Cypher is Neo4j's declarative query language used to create, retrieve, update and delete graph data.

Important Neo4j Cypher Commands

1. Create a Node:

```
CREATE (p:Person {
  name: "John",
  age: 30
})
```

2. Create a Relationship:

```
MATCH (a:Person {name:"John"})
MATCH (b:Person {name:"Alice"})
CREATE (a)-[:FRIEND_OF]->(b)
```

3. Retrieve Nodes:

```
MATCH (p:Person)
RETURN p;
```

4. Update Properties:

```
MATCH (p:Person {name:"John"})
SET p.age = 31;
```

5. Delete a Node:

```
MATCH (p:Person {name:"John"})
DETACH DELETE p;
```

Note: DETACH DELETE removes the node along with all its relationships.

5.5 How Neo4j Differs from Relational Databases

- **Tables vs Graphs:** Relational databases store data in tables using primary and foreign keys and Neo4j stores data as nodes and relationships, enabling faster and simpler queries on connected data.
- **Foreign Keys vs Relationships:** In relational databases, relationships are represented using foreign keys. To retrieve related information, the database must perform JOIN operations across multiple tables.

5.6 Advantages

- Representation of connected data and also represent semi-structured data is easy.
- Its high performance, scalability and flexibility have made it one of the most widely adopted graph databases for handling interconnected data.
- Retrieval or traversal or navigation of connected data is fast and uses simple and powerful data model.

5.7 Limitations

- OLAP support for these types of databases is not well executed.
- In this area, still there are lots of research happening around.

6. Firebase

Firebase is a Google-developed Backend-as-a-Service (BaaS) platform that simplifies backend development for web and mobile applications.

- **Realtime Database:** Stores and syncs NoSQL data in real time.
- **Cloud Firestore:** Scalable NoSQL cloud database.
- **Firebase Authentication:** Secure user authentication with multiple sign-in options.
- **Cloud Functions:** Runs server-side code in response to events.
- **Firebase Cloud Messaging (FCM):** Sends push notifications across platforms.
- **Firebase Hosting:** Fast and secure hosting with built-in SSL.

6.1 Setting Up Firebase

Step 1: Create a Firebase Project

Go to the Firebase Console and click Create a project. Enter a project name, then follow the setup wizard. Enabling Google Analytics is optional.

Step 2: Add the Firebase SDK

After creating the project, select your platform (Web, Android, or iOS) and follow the setup instructions. For a web application, include the required Firebase SDK scripts.

```
<script src="https://www.gstatic.com/firebasejs/8.0.0/firebase-app.js"></script>
<script src="https://www.gstatic.com/firebasejs/8.0.0/firebase-firestore.js"></script>
```

Step 3: Initialize Firebase

Copy your project's Firebase configuration and initialize Firebase in your application.

```
var firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT_ID.firebaseio.com",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT_ID.appspot.com",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

firebase.initializeApp(firebaseConfig);
```

Step 4: Use Firebase Services

After initialization, you can use Firebase services such as Authentication, Cloud Firestore, Cloud Storage, or Cloud Functions.

The following example adds a document to a Firestore collection.

```
var db = firebase.firestore();

db.collection("users").add({
  name: "Raj Kumar",
  email: "raj@example.com"
})
.then(function(docRef) {
  console.log("Document written with ID:", docRef.id);
})
.catch(function(error) {
  console.error("Error adding document:", error);
});
```

Example: Demonstrates the use of Firebase Authentication and Cloud Firestore in a Firebase web application.

```
<!DOCTYPE html>
<html>
<head>
  <title>Firebase Example</title>

  <!-- Firebase SDK -->
  <script src="https://www.gstatic.com/firebasejs/8.0.0/firebase-app.js"></script>
  <script src="https://www.gstatic.com/firebasejs/8.0.0/firebase-auth.js"></script>
  <script src="https://www.gstatic.com/firebasejs/8.0.0/firebase-firestore.js"></script>
</head>

<body>
  <script>
    // Firebase configuration
    var firebaseConfig = {
      apiKey: "YOUR_API_KEY",
      authDomain: "YOUR_PROJECT_ID.firebaseio.com",
      projectId: "YOUR_PROJECT_ID",
      storageBucket: "YOUR_PROJECT_ID.appspot.com",
      messagingSenderId: "YOUR_SENDER_ID",
      appId: "YOUR_APP_ID"
    };

    // Initialize Firebase
    firebase.initializeApp(firebaseConfig);

    // Initialize Firestore
    var db = firebase.firestore();

    // Add data to Firestore
    db.collection("users").add({
      name: "Jane Doe",
      email: "jane@example.com"
    })
    .then(function(docRef) {
      console.log("Document added with ID:", docRef.id);
    })
```

```

        .catch(function(error) {
            console.error("Error adding document:", error);
        });

// Sign in with Firebase Authentication
firebase.auth()
    .signInWithEmailAndPassword("test@example.com", "password123")
    .then(function(userCredential) {
        console.log("User signed in:", userCredential.user);
    })
    .catch(function(error) {
        console.error("Sign-in error:", error);
    });
</script>
</body>
</html>

```

6.2 Firebase Architecture

Firebase is built on Google Cloud and follows a serverless, event-driven architecture that enables scalable and real-time application development.

- **Backend-as-a-Service (BaaS):** Provides backend services through APIs, eliminating backend management.
- **Event-Driven Architecture:** Uses events to automatically trigger Cloud Functions.
- **Real-Time Data Synchronization:** Instantly syncs data across connected clients.

6.3 Advantages of Firebase

- **Serverless Platform:** Eliminates the need to manage or scale backend servers.
- **Simplified Development:** Provides built-in services for authentication, databases, storage, hosting, and Cloud Functions.
- **Real-Time Synchronization:** Keeps data synchronized across connected devices.
- **Cross-Platform Support:** Supports Android, iOS, and web applications with a unified API.
- **Built-in Security:** Offers authentication and security rules to protect user data.
- **Automatic Scalability:** Scales automatically as application traffic grows.

6.4 Use Cases of Firebase

- **Social Media and Chat Apps:** Enables real-time messaging and notifications.
- **Collaborative Applications:** Supports live editing and data synchronization.
- **E-commerce Applications:** Manages user authentication, product catalogs, and orders.
- **Game Development:** Provides backend services, analytics, and performance monitoring.